

ForestMars / Polyglot

🔍

+

<>

Code

🕒

Issues

16

🔗

Pull requests

3

🔗

Agents

💬

Discussions

🎬

Actions

📁

Projects

🛡️

Security and quality

📈

Insights

We are having a problem billing the PreMinder organization. Please [update your payment method](#) or call your payment provider for details on why the transaction failed.

You can always [contact support](#) with any questions.

✔ View status

Code

Preview

eventual coherence v1 #58

Open

ForestMars

wants to merge 11 commits into

main

from

ec1

💬 Conversation

0

🔗 Commits

11

📄 Checks

2

📄 Files changed

11

🔍

Filter files...

▼

src

- ▼ server
 - chatStore.js
 - chatStore.json
 - chatSyncApi.js
- ▼ services
 - api.ts
 - backgroundSync.ts
 - conversationStateManager.ts
 - conversationSync.ts
 - indexedDbStorage.ts
 - ollama.ts
 - App.tsx
 - package.json

▼ src/server/chatStore.js



```

...   @@ -1,5 +1,8 @@
1     // src/server/chatStore.js
2     - // Simple JSON file-based chat store for prototyping

3     import fs from 'fs';
4     import path from 'path';
5     import { fileURLToPath } from 'url';
6     @@ -9,59 +12,180 @@ const __dirname = path.dirname(__filename);
7
8     const STORE_PATH = path.join(__dirname, 'chatStore.json');
9
10    - function readChats() {
11
12    -    if (!fs.existsSync(STORE_PATH)) return [];
13
14    -    const raw = fs.readFileSync(STORE_PATH, 'utf-8');
15    -    try {
16    -        return JSON.parse(raw);
17    -    } catch {
18    -        return [];
19    -    }
20    - }
21
22    - function writeChats(chats) {

```

```

1     // src/server/chatStore.js
2     + // Chat store with eventual coherence: soft-delete, Lamport
3     + // clock, deletion records.
4     + // Hard deletes are gone. A deleted chat stays as
5     + // {is_deleted: true, deletedAtLamport}
6     + // until server-side GC confirms all devices have crossed a
7     + // sync boundary past the deletion.
8     +
9     import fs from 'fs';
10    import path from 'path';
11    import { fileURLToPath } from 'url';
12
13    const STORE_PATH = path.join(__dirname, 'chatStore.json');
14
15    + // -----
16    + // Persistence
17    + // -----
18    +
19    + function readStore() {
20    +     if (!fs.existsSync(STORE_PATH)) return { chats: {},
21    +         lamportClock: 0 };
22    +     const raw = fs.readFileSync(STORE_PATH, 'utf-8');
23    +     try {
24    +         const parsed = JSON.parse(raw);
25    +         // Migrate legacy format (plain array) to store format
26    +         if (Array.isArray(parsed)) {
27    +             const chats = Object.fromEntries(parsed.map(c =>
28    +                 [c.id, c]));
29    +             return { chats, lamportClock: 0 };
30    +         }
31    +         return parsed;
32    +     } catch {
33    +         return { chats: {}, lamportClock: 0 };
34    +     }
35    + function writeStore(store) {

```

```

23 - fs.writeFileSync(STORE_PATH, JSON.stringify(chats, null,
    2));

24 }
25
26 - // Helper function to remove private messages from a chat
    object

27 function filterPrivateMessages(chat) {
28 - if (chat && chat.messages) {
29 -     // Filter out any message where isPrivate is true.

30 -     chat.messages = chat.messages.filter(msg =>
        !msg.isPrivate);
31 - }
32 - return chat;
33 }
34

35 export function getAllChats() {
36 - return readChats();

37 }
38

```

```

36 + fs.writeFileSync(STORE_PATH, JSON.stringify(store, null,
    2));
37 + }
38 +
39 + // -----
    -----
40 + // Lamport clock
41 + // The server clock advances on every write. Returned as
    [logicalTime, 'server'].
42 + // Clients supply their own [logicalTime, deviceId] tuples;
    the server takes
43 + // max(serverClock, clientClock) + 1 on ingestion, per
    Lamport's rule.
44 + // -----
    -----
45 +
46 + function tickClock(store, incomingLamport) {
47 +     const [incomingL] = incomingLamport ?? [0];
48 +     store.lamportClock = Math.max(store.lamportClock,
        incomingL) + 1;
49 +     return [store.lamportClock, 'server'];
50 + }
51 +
52 + // Lexicographic dominance:  $\tau_a > \tau_b$  iff  $a[0] > b[0]$ , or
    ( $a[0] === b[0]$  &&  $a[1] > b[1]$ )
53 + function dominates(tauA, tauB) {
54 +     if (!tauA || !tauB) return false;
55 +     const [lA, dA] = tauA;
56 +     const [lB, dB] = tauB;
57 +     return lA > lB || (lA === lB && String(dA) > String(dB));
58 }
59
60 + // -----
    -----
61 + // Private message filter (preserved from original)
62 + // -----
    -----
63 +
64 function filterPrivateMessages(chat) {
65 + if (chat?.messages) {
66 +     chat.messages = chat.messages.filter(msg =>
        !msg.isPrivate);
67 + }

68 + return chat;

69 }
70
71 + // -----
    -----
72 + // Public API
73 + // -----
    -----
74 +
75 + // Returns all live (non-deleted) chats for normal UI reads.
76 export function getAllChats() {
77 + const { chats } = readStore();
78 + return Object.values(chats).filter(c => !c.is_deleted);
79 }
80

```

```

39   export function addOrUpdateChats(newChats) {
40   -   const chats = readChats();
41   -   const byId = Object.fromEntries(chats.map(c => [c.id,
42     c]));
43   -   for (const chat of newChats) {
44   -     // 🗝 IMPORTANT: Filter private messages from the chat
45     before saving/updating
46   -     const filteredChat = filterPrivateMessages(chat);
47   -     byId[filteredChat.id] = filteredChat;
48
49   }

```

```

81   + // Returns all chats including deletion records, for control
82   plane reconciliation.
83   + export function getAllChatsWithDeleted() {
84   +   const { chats } = readStore();
85   +   return Object.values(chats);
86   + }
87   + // Data plane: add or update chats. Applies Invariant 5: if
88   the server holds a
89   + // deletion record for a chat and the incoming update's
90   Lamport tuple does not
91   + // strictly dominate that record, the update is silently
92   discarded.
93   + // If the incoming update does dominate (causal
94   participation after deletion),
95   + // the resource is restored via upsert semantics (server-
96   level upsert, Definition 6).
97   export function addOrUpdateChats(newChats) {
98   +   const store = readStore();
99   +
100  +   for (const incoming of newChats) {
101  +     // Skip chats without an id
102  +     if (!incoming.id) continue;
103  +
104  +     const filtered = filterPrivateMessages({ ...incoming });
105  +     const existing = store.chats[filtered.id];
106  +
107  +     if (existing?.is_deleted) {
108  +       const tauDelete = existing.deletedAtLamport;
109  +       const tauIncoming = filtered.updatedAtLamport;
110  +
111  +       if (dominates(tauIncoming, tauDelete)) {
112  +         // Device causally participated after deletion:
113  upsert (restore resource).
114  +         // Advance server clock, record the merge.
115  +         const serverTau = tickClock(store, tauIncoming);
116  +         store.chats[filtered.id] = {
117  +           ...filtered,
118  +           is_deleted: false,
119  +           deletedAtLamport: null,
120  +           serverMergedAtLamport: serverTau,
121  +         };
122  +       } else {
123  +         // Normal upsert. Advance server clock.
124  +         const tauIncoming = filtered.updatedAtLamport ?? [0,
125  filtered.id];
126  +         const serverTau = tickClock(store, tauIncoming);
127  +         store.chats[filtered.id] = {
128  +           ...existing,
129  +           ...filtered,
130  +           serverMergedAtLamport: serverTau,
131  +         };
132  +       }
133  +     }
134  +   }

```

```

49 -   writeChats(Object.values(byId));
50   }
51
52 - // NEW: Delete a chat by ID
53 - export function deleteChat(chatId) {
54 -   const chats = readChats();
55 -   const filtered = chats.filter(c => c.id !== chatId);
56 -   writeChats(filtered);
57 -   return filtered.length < chats.length; // Return true if
      something was deleted
131 +   writeStore(store);
132   }
133
134 + // Control plane: mark a chat as deleted on the server.
135 + // Writes a deletion record {is_deleted, deletedAtLamport}
      and retains the entry.
136 + // Returns the deletion record for propagation to clients.
137 + export function deleteChat(chatId, clientLamport) {
138 +   const store = readStore();
139 +   const existing = store.chats[chatId];
140 +
141 +   if (!existing || existing.is_deleted) {
142 +     return null; // Already deleted or never existed.
143 +   }
144 +
145 +   const serverTau = tickClock(store, clientLamport);
146 +   store.chats[chatId] = {
147 +     ...existing,
148 +     is_deleted: true,
149 +     deletedAtLamport: serverTau,
150 +     // Strip content to minimise storage; retain id and
      topology metadata.
151 +     messages: undefined,
152 +   };
153 +
154 +   writeStore(store);
155 +   return { id: chatId, deletedAtLamport: serverTau };
156 + }
157 +
158 + // Control plane: bulk delete.
159 + export function deleteChats(chatIds, clientLamport) {
160 +   const store = readStore();
161 +   const records = [];
162 +
163 +   for (const chatId of chatIds) {
164 +     const existing = store.chats[chatId];
165 +     if (!existing || existing.is_deleted) continue;
166 +
167 +     const serverTau = tickClock(store, clientLamport);
168 +     store.chats[chatId] = {
169 +       ...existing,
170 +       is_deleted: true,
171 +       deletedAtLamport: serverTau,
172 +       messages: undefined,
173 +     };
174 +     records.push({ id: chatId, deletedAtLamport: serverTau
      });
175 +   }
176 +
177 +   writeStore(store);
178 +   return records;
179   }
180
181 + // GC: hard-delete a chat only after all known devices have
      acknowledged the
182 + // deletion by crossing a sync boundary past
      deletedAtLamport.
183 + // Call this from a separate GC job, not from the sync path.
184 + export function gcChat(chatId) {
185 +   const store = readStore();

```

<pre> 65 - writeChats(filtered); 66 - return chats.length - filtered.length; // Return count of deleted chats 67 } </pre>	<pre> 186 + const existing = store.chats[chatId]; 187 + if (!existing?.is_deleted) return false; 188 + delete store.chats[chatId]; 189 + writeStore(store); 190 + return true; 191 } </pre>
--	--

src/server/chatStore.json

Load Diff

Large diffs are not rendered by default.

src/server/chatSyncApi.js

<pre> ... @@ -1,16 +1,32 @@ 1 // src/server/chatSyncApi.js 2 - // Minimal REST API for chat sync, no Express required. 3 4 import http from 'http'; 5 - import { getAllChats, addOrUpdateChats, deleteChat, deleteChats } from './chatStore.js'; 6 7 const PORT = process.env.CHAT_SYNC_PORT 4001; 8 9 - // Utility: parse JSON body 10 11 function parseBody(req) { 12 return new Promise((resolve, reject) => { 13 let data = ''; 14 req.on('data', chunk => data += chunk); 15 req.on('end', () => { 16 try { 17 resolve(JSON.parse(data '{}')); 18 } catch { 19 reject(new Error('Invalid JSON')); 20 } 21 }); 22 }); 23 } </pre>	<pre> 1 // src/server/chatSyncApi.js 2 + // Chat sync API with control/data plane separation. 3 + // 4 + // Data plane → POST /pushChats real-time update propagation, Invariant 5 enforced in chatStore 5 + // Control plane → POST /sync reconciliation at epoch boundaries, deletion records exchanged 6 + // → POST /deleteChat, POST /deleteChats initiate deletions (write deletion records) 7 + // 8 + // The old DELETE /deleteChat/:id is gone. Deletion is always a control plane write, 9 + // never a hard remove at the HTTP layer. 10 11 import http from 'http'; 12 + import { 13 + getAllChats, 14 + getAllChatsWithDeleted, 15 + addOrUpdateChats, 16 + deleteChat, 17 + deleteChats, 18 + } from './chatStore.js'; 19 20 const PORT = process.env.CHAT_SYNC_PORT 4001; 21 22 + // ----- 23 + // Utility 24 + // ----- 25 + 26 function parseBody(req) { 27 return new Promise((resolve, reject) => { 28 let data = ''; 29 req.on('data', chunk => (data += chunk)); 30 req.on('end', () => { 31 try { 32 resolve(JSON.parse(data '{}')); 33 } catch { 34 reject(new Error('Invalid JSON')); 35 } 36 }); 37 }); </pre>
--	---

```

22     }
23
24     - // Create server

```

```

38     }
39
40     + function json(res, status, body) {
41     +   res.writeHead(status, { 'Content-Type': 'application/json'
42     +     });
43     +   res.end(JSON.stringify(body));
44     + }
45     + // Lexicographic dominance (mirrors chatStore.js - kept
46     +   local to avoid a shared
47     + // util import for now; extract to shared/lamport.js when
48     +   the codebase grows).
49     + function dominates(tauA, tauB) {
50     +   if (!tauA || !tauB) return false;
51     +   const [lA, dA] = tauA;
52     +   const [lB, dB] = tauB;
53     +   return lA > lB || (lA === lB && String(dA) > String(dB));
54     + }
55     + // -----
56     + // Reconciliation (control plane, Section 3.4 Case 1)
57     + // Client sends its full state: live chats + deletion
58     +   records.
59     + // Server applies the causal participation check and
60     +   returns:
61     + //   - chats the client is missing (server has, client
62     +     doesn't)
63     + //   - deletion records the client hasn't seen
64     + //   - chats the server accepted from the client's push
65     + // The check for each server-held deletion record d against
66     +   a client chat c:
67     + //   If c.updatedAtLamport < d.deletedAtLamport, c has no
68     +     causal participation
69     + //   after deletion → client must accept the deletion.
70     + //   If c.updatedAtLamport > d.deletedAtLamport, c causally
71     +     participated after
72     + //   deletion → server upserts c (handled in
73     +     addOrUpdateChats), deletion loses.
74     + // -----
75     +
76     + function reconcile(clientChats, clientDeletionRecords) {
77     +   // Ingest client's live chats (chatStore enforces
78     +     Invariant 5 internally)
79     +   if (clientChats.length > 0) {
80     +     addOrUpdateChats(clientChats);
81     +   }
82     +   const serverAll = getAllChatsWithDeleted();
83     +   const serverById = Object.fromEntries(serverAll.map(c =>
84     +     [c.id, c]));
85     +   const clientLiveIds = new Set(clientChats.map(c => c.id));
86     +   const clientDeletedIds = new Set((clientDeletionRecords ||
87     +     []).map(r => r.id));
88     +   // Chats the client has live that the server has marked
89     +     deleted:

```

```

83 + // already handled in addOrUpdateChats (discarded unless
      causal dominance).
84 + // We need to tell the client about server-side deletions
      it hasn't seen.
85 + const deletionsForClient = serverAll
86 +   .filter(c => c.is_deleted &&
      !clientDeletedIds.has(c.id))
87 +   .map(c => ({ id: c.id, deletedAtLamport:
      c.deletedAtLamport }));
88 +
89 + // Chats the client is missing entirely (server has live,
      client has neither
90 + // live nor a deletion record for them – topologically
      unknown resource case).
91 + const missingForClient = serverAll.filter(
92 +   c => !c.is_deleted && !clientLiveIds.has(c.id) &&
      !clientDeletedIds.has(c.id)
93 + );
94 +
95 + // Deletion records the client sent that the server
      doesn't have yet.
96 + // Apply them now (control plane delete on behalf of
      client).
97 + const clientInitiatedDeletes = (clientDeletionRecords ||
      []).filter(
98 +   r => !serverById[r.id]?.is_deleted
99 + );
100 + if (clientInitiatedDeletes.length > 0) {
101 +   deleteChats(
102 +     clientInitiatedDeletes.map(r => r.id),
103 +     clientInitiatedDeletes[0]?.deletedAtLamport
104 +   );
105 + }
106 +
107 + return { missing: missingForClient, deletions:
      deletionsForClient };
108 + }
109 +
110 + // -----
      -----
111 + // Server
112 + // -----
      -----
113 +
114 const server = http.createServer(async (req, res) => {
115   res.setHeader('Access-Control-Allow-Origin', '*');
116 + res.setHeader('Access-Control-Allow-Methods', 'GET, POST,
      OPTIONS');
117   res.setHeader('Access-Control-Allow-Headers', 'Content-
      Type');
118
119   if (req.method === 'OPTIONS') {
120     res.status(200).end();
121   }
122
123   try {
124     // -----
      -----
125
126 + // GET /fetchChats → live chats only (data plane read,
      no deletion records)

```

```

38     if (req.method === 'GET' && req.url === '/fetchChats') {
39 -     res.writeHead(200, { 'Content-Type':
      'application/json' });
40 -     return res.end(JSON.stringify(getAllChats()));
41   }
42
43 -   // POST /pushChats or /sync → add/update chats
44 -   if (req.method === 'POST' && (req.url === '/pushChats'
      || req.url === '/sync')) {
45 -     const body = await parseBody(req);
46 -     const { chats } = body;
47
48 -     if (!Array.isArray(chats)) {
49 -       res.writeHead(400, { 'Content-Type':
        'application/json' });
50 -       return res.end(JSON.stringify({ error: 'chats must
          be array' }));
51 -     }
52 -
53 -     // Add or update incoming chats
54     addOrUpdateChats(chats);
55
56 -     // If syncing, return chats missing on the client
57 -     if (req.url === '/sync') {
58 -       const serverChats = getAllChats();
59 -       const clientIds = new Set(chats.map(c => c.id));
60 -       const missing = serverChats.filter(c =>
        !clientIds.has(c.id));
61 -       res.writeHead(200, { 'Content-Type':
        'application/json' });
62 -       return res.end(JSON.stringify({ missing }));
63 -     }
64 -
65 -     // POST /pushChats response
66 -     res.writeHead(200, { 'Content-Type':
      'application/json' });
67 -     return res.end(JSON.stringify({ ok: true }));
68   }
69
70 -   // DELETE /deleteChat/:id → delete a single chat
71 -   if (req.method === 'DELETE' &&
      req.url?.startsWith('/deleteChat/')) {
72 -     const chatId = req.url.split('/deleteChat/')[1];
127 +   // -----
128     if (req.method === 'GET' && req.url === '/fetchChats') {
129 +     return json(res, 200, getAllChats());
130   }
131
132 +   // -----
133 +   // POST /pushChats → data plane: real-time update
      propagation.
134 +   // Invariant 5 enforced in chatStore.addOrUpdateChats.
135 +   // No reconciliation; no deletion records returned.
136 +   // -----
137 +   if (req.method === 'POST' && req.url === '/pushChats') {
138 +     const { chats } = await parseBody(req);
139 +     if (!Array.isArray(chats)) return json(res, 400, {
      error: 'chats must be array' });
140
141     addOrUpdateChats(chats);
142 +     return json(res, 200, { ok: true });
143 +   }
144
145 +   // -----
146 +   // POST /sync → control plane: epoch-boundary
      reconciliation.
147 +   // Client sends { chats, deletionRecords }.
148 +   // Server returns { missing, deletions }.
149 +   // -----
150 +   if (req.method === 'POST' && req.url === '/sync') {
151 +     const { chats, deletionRecords } = await
      parseBody(req);
152 +     if (!Array.isArray(chats)) return json(res, 400, {
      error: 'chats must be array' });
153 +
154 +     const { missing, deletions } = reconcile(chats,
      deletionRecords || []);
155 +     return json(res, 200, { missing, deletions });
156   }
157
158 +   // -----
159 +   // POST /deleteChat → control plane: single deletion
      record write.
160 +   // Body: { chatId, lamport }

```

```

73 -     if (!chatId) {
74 -         res.writeHead(400, { 'Content-Type':
75 -             'application/json' });
76 -         return res.end(JSON.stringify({ error: 'Chat ID
77 -             required' }));
78 -     }
79 -     const deleted = deleteChat(chatId);
80 -     res.writeHead(200, { 'Content-Type':
81 -         'application/json' });
82 -     return res.end(JSON.stringify({ ok: true, deleted }));
83 - }
84 - // POST /deleteChats → delete multiple chats
85 -
86 -
87 -
88 -
89 -
90 -
91 -
92 -
93 -
94 -
95 -
96 -
97 -
98 - // Fallback 404
99 - res.writeHead(404, { 'Content-Type': 'application/json'
100 - });
101 - res.end(JSON.stringify({ error: 'Not found' }));
102 - } catch (err) {
103 - res.writeHead(500, { 'Content-Type': 'application/json'
104 - });
105 - res.end(JSON.stringify({ error: err.message }));
106 - }
107 - };
108 - };
109 - // Only start the server if this file is run directly
110 - if (import.meta.url === `file://${process.argv[1]}`) {
111 -     server.listen(PORT, () => {
112 -         console.log(`Chat sync API running on
113 -             http://localhost:${PORT}`);
114 -     });
115 - }
116 -
117 -
118 -
119 -
120 -
121 -
122 -
123 -
124 -
125 -
126 -
127 -
128 -
129 -
130 -
131 -
132 -
133 -
134 -
135 -
136 -
137 -
138 -
139 -
140 -
141 -
142 -
143 -
144 -
145 -
146 -
147 -
148 -
149 -
150 -
151 -
152 -
153 -
154 -
155 -
156 -
157 -
158 -
159 -
160 -
161 + // -----
162 +
163 +
164 +
165 +
166 +
167 +
168 +
169 +
170 + // -----
171 + // POST /deleteChats → control plane: bulk deletion
172 + // Body: { chatIds, lamport }
173 + // -----
174 +
175 +
176 +
177 +
178 +
179 +
180 +
181 +
182 +
183 +
184 +
185 +
186 +
187 +
188 +
189 +
190 +
191 +
192 +
193 +
194 +
195 +
196 +
197 +
198 +
199 +
200 +
201 +
202 +
203 +
204 +
205 +
206 +
207 +
208 +
209 +
210 +
211 +
212 +
213 +
214 +
215 +
216 +
217 +
218 +
219 +
220 +
221 +
222 +
223 +
224 +
225 +
226 +
227 +
228 +
229 +
230 +
231 +
232 +
233 +
234 +
235 +
236 +
237 +
238 +
239 +
240 +
241 +
242 +
243 +
244 +
245 +
246 +
247 +
248 +
249 +
250 +
251 +
252 +
253 +
254 +
255 +
256 +
257 +
258 +
259 +
260 +
261 +
262 +
263 +
264 +
265 +
266 +
267 +
268 +
269 +
270 +
271 +
272 +
273 +
274 +
275 +
276 +
277 +
278 +
279 +
280 +
281 +
282 +
283 +
284 +
285 +
286 +
287 +
288 +
289 +
290 +
291 +
292 +
293 +
294 +
295 +
296 +
297 +
298 +
299 +
300 +
301 +
302 +
303 +
304 +
305 +
306 +
307 +
308 +
309 +
310 +
311 +
312 +
313 +
314 +
315 +
316 +
317 +
318 +
319 +
320 +
321 +
322 +
323 +
324 +
325 +
326 +
327 +
328 +
329 +
330 +
331 +
332 +
333 +
334 +
335 +
336 +
337 +
338 +
339 +
340 +
341 +
342 +
343 +
344 +
345 +
346 +
347 +
348 +
349 +
350 +
351 +
352 +
353 +
354 +
355 +
356 +
357 +
358 +
359 +
360 +
361 +
362 +
363 +
364 +
365 +
366 +
367 +
368 +
369 +
370 +
371 +
372 +
373 +
374 +
375 +
376 +
377 +
378 +
379 +
380 +
381 +
382 +
383 +
384 +
385 +
386 +
387 +
388 +
389 +
390 +
391 +
392 +
393 +
394 +
395 +
396 +
397 +
398 +
399 +
400 +
401 +
402 +
403 +
404 +
405 +
406 +
407 +
408 +
409 +
410 +
411 +
412 +
413 +
414 +
415 +
416 +
417 +
418 +
419 +
420 +
421 +
422 +
423 +
424 +
425 +
426 +
427 +
428 +
429 +
430 +
431 +
432 +
433 +
434 +
435 +
436 +
437 +
438 +
439 +
440 +
441 +
442 +
443 +
444 +
445 +
446 +
447 +
448 +
449 +
450 +
451 +
452 +
453 +
454 +
455 +
456 +
457 +
458 +
459 +
460 +
461 +
462 +
463 +
464 +
465 +
466 +
467 +
468 +
469 +
470 +
471 +
472 +
473 +
474 +
475 +
476 +
477 +
478 +
479 +
480 +
481 +
482 +
483 +
484 +
485 +
486 +
487 +
488 +
489 +
490 +
491 +
492 +
493 +
494 +
495 +
496 +
497 +
498 +
499 +
500 +
501 +
502 +
503 +
504 +
505 +
506 +
507 +
508 +
509 +
510 +
511 +
512 +
513 +
514 +
515 +
516 +
517 +
518 +
519 +
520 +
521 +
522 +
523 +
524 +
525 +
526 +
527 +
528 +
529 +
530 +
531 +
532 +
533 +
534 +
535 +
536 +
537 +
538 +
539 +
540 +
541 +
542 +
543 +
544 +
545 +
546 +
547 +
548 +
549 +
550 +
551 +
552 +
553 +
554 +
555 +
556 +
557 +
558 +
559 +
560 +
561 +
562 +
563 +
564 +
565 +
566 +
567 +
568 +
569 +
570 +
571 +
572 +
573 +
574 +
575 +
576 +
577 +
578 +
579 +
580 +
581 +
582 +
583 +
584 +
585 +
586 +
587 +
588 +
589 +
590 +
591 +
592 +
593 +
594 +
595 +
596 +
597 +
598 +
599 +
600 +
601 +
602 +
603 +
604 +
605 +
606 +
607 +
608 +
609 +
610 +
611 +
612 +
613 +
614 +
615 +
616 +
617 +
618 +
619 +
620 +
621 +
622 +
623 +
624 +
625 +
626 +
627 +
628 +
629 +
630 +
631 +
632 +
633 +
634 +
635 +
636 +
637 +
638 +
639 +
640 +
641 +
642 +
643 +
644 +
645 +
646 +
647 +
648 +
649 +
650 +
651 +
652 +
653 +
654 +
655 +
656 +
657 +
658 +
659 +
660 +
661 +
662 +
663 +
664 +
665 +
666 +
667 +
668 +
669 +
670 +
671 +
672 +
673 +
674 +
675 +
676 +
677 +
678 +
679 +
680 +
681 +
682 +
683 +
684 +
685 +
686 +
687 +
688 +
689 +
690 +
691 +
692 +
693 +
694 +
695 +
696 +
697 +
698 +
699 +
700 +
701 +
702 +
703 +
704 +
705 +
706 +
707 +
708 +
709 +
710 +
711 +
712 +
713 +
714 +
715 +
716 +
717 +
718 +
719 +
720 +
721 +
722 +
723 +
724 +
725 +
726 +
727 +
728 +
729 +
730 +
731 +
732 +
733 +
734 +
735 +
736 +
737 +
738 +
739 +
740 +
741 +
742 +
743 +
744 +
745 +
746 +
747 +
748 +
749 +
750 +
751 +
752 +
753 +
754 +
755 +
756 +
757 +
758 +
759 +
760 +
761 +
762 +
763 +
764 +
765 +
766 +
767 +
768 +
769 +
770 +
771 +
772 +
773 +
774 +
775 +
776 +
777 +
778 +
779 +
780 +
781 +
782 +
783 +
784 +
785 +
786 +
787 +
788 +
789 +
790 +
791 +
792 +
793 +
794 +
795 +
796 +
797 +
798 +
799 +
800 +
801 +
802 +
803 +
804 +
805 +
806 +
807 +
808 +
809 +
810 +
811 +
812 +
813 +
814 +
815 +
816 +
817 +
818 +
819 +
820 +
821 +
822 +
823 +
824 +
825 +
826 +
827 +
828 +
829 +
830 +
831 +
832 +
833 +
834 +
835 +
836 +
837 +
838 +
839 +
840 +
841 +
842 +
843 +
844 +
845 +
846 +
847 +
848 +
849 +
850 +
851 +
852 +
853 +
854 +
855 +
856 +
857 +
858 +
859 +
860 +
861 +
862 +
863 +
864 +
865 +
866 +
867 +
868 +
869 +
870 +
871 +
872 +
873 +
874 +
875 +
876 +
877 +
878 +
879 +
880 +
881 +
882 +
883 +
884 +
885 +
886 +
887 +
888 +
889 +
890 +
891 +
892 +
893 +
894 +
895 +
896 +
897 +
898 +
899 +
900 +
901 +
902 +
903 +
904 +
905 +
906 +
907 +
908 +
909 +
910 +
911 +
912 +
913 +
914 +
915 +
916 +
917 +
918 +
919 +
920 +
921 +
922 +
923 +
924 +
925 +
926 +
927 +
928 +
929 +
930 +
931 +
932 +
933 +
934 +
935 +
936 +
937 +
938 +
939 +
940 +
941 +
942 +
943 +
944 +
945 +
946 +
947 +
948 +
949 +
950 +
951 +
952 +
953 +
954 +
955 +
956 +
957 +
958 +
959 +
960 +
961 +
962 +
963 +
964 +
965 +
966 +
967 +
968 +
969 +
970 +
971 +
972 +
973 +
974 +
975 +
976 +
977 +
978 +
979 +
980 +
981 +
982 +
983 +
984 +
985 +
986 +
987 +
988 +
989 +
990 +
991 +
992 +
993 +
994 +
995 +
996 +
997 +
998 +
999 +

```

114

- // Export for testing

115

export { server };

⊖

194

export { server };

⊖

src/services/api.ts

⋮

Load Diff

Large diffs are not rendered by default.

src/services/backgroundSync.ts

⋮

Load Diff

Large diffs are not rendered by default.

src/services/conversationStateManager.ts

⋮

Load Diff

Large diffs are not rendered by default.

src/services/conversationSync.ts

⋮

⋮ @@ -0,0 +1,67 @@

1 + // src/services/conversationSync.ts

2 + //

3 + // Coordinator: single call site for any operation that touches both local

4 + // storage and the sync layer. Replaces the direct fetch calls that were

5 + // previously embedded in indexedDbStorage.saveConversation and

6 + // indexedDbStorage.deleteConversation.

7 + //

8 + // Usage (replaces old storage.saveConversation / storage.deleteConversation):

9 + //

10 + // import { saveConversation, deleteConversation } from './conversationSync';

11 + // await saveConversation(chat);

12 + // await deleteConversation(id);

13 +

14 + import { storage, Chat } from './indexedDbStorage';

15 + import { backgroundSync } from './backgroundSync';

16 +

17 + // Save a conversation locally and propagate to server via data plane.

18 + // If the chat has no non-private messages after filtering, delete it

19 + // both locally and on the server (preserving the original behaviour).

20 + export async function saveConversation(chat: Chat): Promise<void> {

```

21 +   const filteredMessages = (chat.messages || []).filter(m =>
      !m.isPrivate);
22 +
23 +   if (filteredMessages.length === 0) {
24 +     // Nothing to persist – treat as a delete if it
      previously existed.
25 +     if (chat.id) {
26 +       await deleteConversation(chat.id);
27 +     }
28 +     return;
29 +   }
30 +
31 +   const toSave: Chat = { ...chat, messages: filteredMessages
      };
32 +
33 +   // Write locally first.
34 +   await storage.saveChat(toSave);
35 +
36 +   // Propagate to server with Lamport stamp. Fire-and-forget:
      local write
37 +   // already succeeded; server will catch up at next sync
      boundary if push fails.
38 +   backgroundSync.pushChat(toSave).catch(err =>
39 +     console.warn('[conversationSync] Push failed, will retry
      at sync boundary:', err)
40 +   );
41 + }
42 +
43 + // Delete a conversation locally and register a deletion
      record.
44 + // The deletion record is what prevents the chat from being
      restored on next sync.
45 + export async function deleteConversation(id: string):
      Promise<void> {
46 +   // Write deletion record and push to server (backgroundSync
      handles both).
47 +   // Do this before local delete so the record exists if
      anything throws.
48 +   await backgroundSync.pushDeletion(id);
49 +
50 +   // Remove from local IndexedDB.
51 +   try {
52 +     await storage.deleteChat(id);
53 +   } catch (err) {
54 +     // Already absent locally – deletion record is still
      registered, so
55 +     // Invariant 5 holds. Log and continue.
56 +     console.warn(`[conversationSync] Local delete failed for
      ${id} (may not exist):`, err);
57 +   }
58 +
59 +   // Notify UI.
60 +   if (typeof window !== 'undefined') {
61 +     try {
62 +       window.dispatchEvent(new Event('conversations-
      updated'));
63 +     } catch (e) {
64 +       console.warn('[conversationSync] Failed to dispatch
      conversations-updated:', e);
65 +     }
66 +   }

```

67 + }



src/services/indexedDbStorage.ts



Load Diff

Large diffs are not rendered by default.

```

... @@ -1,5 +1,5 @@
1   export interface OllamaMessage {
2   -   role: 'user' | 'assistant' | 'system';
3     content: string;
4   }
5
6   @@ -24,102 +24,119 @@ export interface OllamaResponse {
24     content: string;
25   };
26   done: boolean;
27   - total_duration?: number;          // Total time in
    nanoseconds
28   - load_duration?: number;          // Model loading time
29   - prompt_eval_count?: number;      // Tokens processed
30   - prompt_eval_duration?: number;   // Prompt processing time
31   - eval_count?: number;             // Response tokens
32   - eval_duration?: number;          // Response generation
    time
33   }
34
35   export class OllamaService {
36     private baseUrl: string;
37
38   -   constructor(baseUrl: string = 'http://localhost:11434') {
39       this.baseUrl = baseUrl;
40     }
41
42     async chat(request: OllamaRequest):
    Promise<OllamaResponse> {
43   -     try {
44   -       const response = await
    fetch(`${this.baseUrl}/api/chat`, {
45   -         method: 'POST',
46   -         headers: {
47   -           'Content-Type': 'application/json',
48   -         },
49   -         body: JSON.stringify(request),
50   -       });
51
52   -       if (!response.ok) {
53   -         throw new Error(`Ollama API error:
    ${response.status} ${response.statusText}`);
54   -       }
55
56   -       return await response.json();
57   -     } catch (error) {
1   export interface OllamaMessage {
2   +   role: "user" | "assistant" | "system";
3     content: string;
4   }
5
24     content: string;
25   };
26   done: boolean;
27   + total_duration?: number; // Total time in nanoseconds
28   + load_duration?: number; // Model loading time
29   + prompt_eval_count?: number; // Tokens processed
30   + prompt_eval_duration?: number; // Prompt processing time
31   + eval_count?: number; // Response tokens
32   + eval_duration?: number; // Response generation time
33   }
34
35   export class OllamaService {
36     private baseUrl: string;
37
38   +   constructor(baseUrl: string = "http://localhost:11434") {
39       this.baseUrl = baseUrl;
40     }
41
42     async chat(request: OllamaRequest):
    Promise<OllamaResponse> {
43   +     const response = await fetch(`${this.baseUrl}/api/chat`,
    {
44   +       method: "POST",
45   +       headers: {
46   +         "Content-Type": "application/json",
47   +       },
48   +       body: JSON.stringify(request),
49   +     });
50
51   +     if (!response.ok) {
52   +       const errorBody = await response.text().catch(() => "
    <no body>");
53   +       console.error(
54   +         `[OllamaService] chat failed: ${response.status} -
    ${errorBody}`,
55   +       );
56   +       throw new Error(

```

```

58 -     console.error('Ollama API error:', error);
59 -     throw error;
60     }
61 }
62
63 - async listModels(): Promise<{ models: Array<{ name:
    string; modified_at: string; size: number }> }> {
64     try {
65         const response = await
        fetch(`${this.baseUrl}/api/tags`);
66 -
67         if (!response.ok) {
68 -             throw new Error(`Failed to fetch models:
                ${response.status} ${response.statusText}`);
69         }
70
71         return await response.json();
72     } catch (error) {
73 -         console.error('Failed to fetch Ollama models:',
            error);
74         throw error;
75     }
76 }
77
78 - async generate(request: Omit<OllamaRequest, 'messages'> &
    { prompt: string }): Promise<OllamaResponse> {
79     try {
80         const response = await
        fetch(`${this.baseUrl}/api/generate`, {
81 -             method: 'POST',
82             headers: {
83 -                 'Content-Type': 'application/json',
84             },
85             body: JSON.stringify(request),
86         });
87
88         if (!response.ok) {
89 -             throw new Error(`Ollama API error:
                ${response.status} ${response.statusText}`);
90         }
91

```

```

57 +         `Ollama API error: ${response.status}
            ${response.statusText} - ${errorBody}`,
58 +     );
59     }
60 +
61 +     return await response.json();
62     }
63
64 + async listModels(): Promise<{
65 +     models: Array<{ name: string; modified_at: string; size:
        number }>;
66 + }> {
67     try {
68         const response = await
        fetch(`${this.baseUrl}/api/tags`);
69 +
70         if (!response.ok) {
71 +             throw new Error(
72 +                 `Failed to fetch models: ${response.status}
                    ${response.statusText}`,
73 +             );
74         }
75
76         return await response.json();
77     } catch (error) {
78 +         console.error("Failed to fetch Ollama models:",
            error);
79         throw error;
80     }
81 }
82
83 + async generate(
84 +     request: Omit<OllamaRequest, "messages"> & { prompt:
        string },
85 + ): Promise<OllamaResponse> {
86     try {
87         const response = await
        fetch(`${this.baseUrl}/api/generate`, {
88 +             method: "POST",
89             headers: {
90 +                 "Content-Type": "application/json",
91             },
92             body: JSON.stringify(request),
93         });
94
95         if (!response.ok) {
96 +             const errorBody = await response.text().catch(() =>
                "<no body>");
97 +             console.error(
98 +                 `[OllamaService] chat failed: ${response.status} -
                    ${errorBody}`,
99 +             );
100 +             throw new Error(
101 +                 `Ollama API error: ${response.status}
                    ${response.statusText} - ${errorBody}`,
102 +             );
103         }
104

```

```

92         return await response.json();
93     } catch (error) {
94 -       console.error('Ollama API error:', error);
95       throw error;
96     }
97   }
98
99   async healthCheck(): Promise<boolean> {
100     const url = `${this.baseUrl}/api/tags`;
101     console.log(`[OllamaService] Checking health at:
      ${url}`);
102 -
103     try {
104       const startTime = Date.now();
105       const response = await fetch(url, {
106 -         method: 'GET',
107         headers: {
108 -         'Accept': 'application/json',
109         },
110       });
111       const endTime = Date.now();
112 -
113 -       console.log(`[OllamaService] Health check response:
      ${response.status} ${response.statusText} (${endTime -
      startTime}ms)`);
114 -
115       if (!response.ok) {
116 -         const errorText = await response.text().catch(() =>
      'No error details');
117 -         console.error(`[OllamaService] Health check failed:
      ${response.status} ${response.statusText} - ${errorText}`);
118       }
119 -
120       return response.ok;
121     } catch (error) {
122 -       console.error('[OllamaService] Health check error:',
      error);
123       return false;
124     }
125   }
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

... @@ -15,23 +15,17 @@ const queryClient = new QueryClient();
15
16 const App = () => {
17   console.log('App component is running');
18 -
19 -
20 - // Polyfill for requestIdleCallback
21 - const rIC = (cb: Function) => (window as
      any).requestIdleCallback ? (window as

```

<pre> any).requestIdleCallback(cb) : setTimeout(cb, 200); 22 - 23 - console.log('App component is running'); 24 25 useEffect(() => { 26 - // 1. Render UI immediately (this component) 27 - // 2. Schedule background tasks after paint 28 rIC(async () => { 29 - // Task A: Fast load of initial conversations for UI (sidebar, preview) 30 try { 31 - await indexedDbStorage.ready; 32 - // Example: load top 20 conversations (headers only) 33 34 const items = await indexedDbStorage.listConversations({ limit: 20 }); 35 // TODO: set into UI store or state for sidebar/preview 36 console.log('[startup] Loaded initial conversations:', items.length); 37 } catch (err) { 38 console.error('[startup] Failed to load initial conversations', err); 39 } 40 }); 41 42 // Still run MCP service init 43 mcpService.initialize().catch((error) => { 44 console.error('Failed to initialize MCP service:', error); 45 }); 46 47 @@ -56,7 +50,6 @@ const App = () => { 48 } 49 }); 50 51 // Still run MCP service init 52 mcpService.initialize().catch((error) => { 53 console.error('Failed to initialize MCP service:', error); 54 }); 55 56 @@ -78,4 +71,4 @@ const App = () => { 57 }; 58 }; 59 60 - export default App; </pre>	<pre> : setTimeout(cb, 200); 21 + 22 23 useEffect(() => { 24 25 rIC(async () => { 26 // Task A: Load initial conversations for UI (sidebar, preview) 27 try { 28 + await indexedDbStorage.initialize(); 29 + const items = await indexedDbStorage.listConversations(false); 30 31 console.log('[startup] Loaded initial conversations:', items.length); 32 } catch (err) { 33 console.error('[startup] Failed to load initial conversations', err); 34 35 } 36 }); 37 38 mcpService.initialize().catch((error) => { 39 console.error('Failed to initialize MCP service:', error); 40 }); 41 42 @@ -71,4 +71,4 @@ const App = () => { 43 }; 44 }; 45 46 + export default App; </pre>
---	---



package.json



<pre> @@ -5,6 +5,9 @@ 5 "type": "module", 6 "scripts": { 7 "dev": "vite", 8 9 "build": "tsc --noEmit && vite build", 10 "build:ci": "npm run build && npm run test:ci", 11 "build:dev": "vite build --mode development", </pre>	<pre> 5 "type": "module", 6 "scripts": { 7 "dev": "vite", 8 + "app": "bun -v", 9 + "chat": "bun src/server/chatSyncApi.js", 10 + "chat:test": "npm test -- src/tests/server/chatSync.unit.test.ts", 11 "build": "tsc --noEmit && vite build", 12 "build:ci": "npm run build && npm run test:ci", 13 "build:dev": "vite build --mode development", </pre>
---	---